

UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE COMERCIO Y TURISMO
NTIC MASTER

Máster en Data Science, Big Data & Business Analytics



TRABAJO DE FIN DE MASTER

MODELOS DE LENGUAJE APLICADOS A LA PREDICCIÓN DE ACORDES:
ENFOQUE ESTADÍSTICO VS REDES NEURONALES

*LANGUAGE MODELS APPLIED TO CHORD PREDICTION:
STATISTICAL APPROACH VS NEURAL NETWORKS*

Antonio Luis Martínez Trapote

Curso académico: 2024-25
Convocatoria ordinaria

Resumen

Este trabajo presenta el desarrollo de un asistente de composición musical basado en la predicción de acordes a partir de progresiones armónicas. Partiendo del corpus público *Jazz Chord Progressions* (2.612 canciones, ~134.000 acordes), se construyó un pipeline *end-to-end* que incluyó: análisis exploratorio, transcripción a notación funcional, entrenamiento de modelos estadísticos y neuronales, y despliegue en una API con FastAPI.

Se evaluaron enfoques estadísticos y neuronales. El modelo interpolado de Kneser–Ney ofreció un rendimiento competitivo y estable, superando en la práctica a arquitecturas recurrentes (GRU, LSTM), pese a que estas alcanzaron métricas ligeramente mejores en validación. La evaluación cualitativa mostró que Kneser–Ney genera sugerencias más plausibles y explicables. Además, se incorporó un módulo de re-ranking que penaliza repeticiones y filtra acordes según distintos grados de restricción tonal, mejorando la utilidad práctica del sistema.

El proyecto demuestra que un modelo ligero y explicable puede resultar más efectivo que arquitecturas complejas en tareas musicales aplicadas. Se concluye que es posible construir un asistente de predicción armónica eficiente combinando teoría musical y técnicas de NLP, abriendo la puerta a futuras mejoras como la incorporación de nuevos géneros, segmentación tonal más fina o arquitecturas modernas como transformers.

Abstract

This paper presents the development of a music composition assistant based on chord prediction from harmonic progressions. Starting with the Jazz Chord Progressions public corpus (2,612 songs, ~134,000 chords), an end-to-end pipeline was constructed. This included: exploratory data analysis, transcription to functional notation, training of statistical and neural models, and deployment in an API with FastAPI.

Statistical and neural approaches were evaluated. In practice, the Kneser–Ney interpolated model offered competitive and stable performance, outperforming recurrent architectures (GRU and LSTM), even though the latter achieved slightly better validation metrics. Qualitative evaluation showed the Kneser–Ney model generates more plausible and explainable suggestions. Additionally, a re-ranking module was incorporated that penalises repetitions and filters chords according to different degrees of tonal restriction, thereby improving the system's practical usefulness.

This project demonstrates that a lightweight, explainable models can be more effective than complex architectures for applied musical tasks. It can be concluded that an efficient harmonic prediction assistant can be built by combining music theory and NLP techniques, paving the way for future enhancements such a incorporating new genres, achieving finer tonal segmentation, or modern architectures such as transformers.

Índice

1. Introducción y objetivos
2. Origen y licencia del *dataset*.
3. Análisis exploratorio de los datos
 - 3.1. Tonalidades
 - 3.2. Métrica
 - 3.3. Compases por canción
 - 3.4. Acordes
4. Tokenización de los acordes: de cifrado americano a notación funcional
 - 4.1. Parseo y normalización de acordes
 - 4.2. Estimación de tonalidad
 - 4.3. Transcripción de acordes a notación funcional
 - 4.3.1. Acordes diatónicos
 - 4.3.2. Préstamos habituales
 - 4.3.3. Dominantes secundarios y sustitutos
 - 4.3.4. Ejemplo completo
 - 4.4. Transcripción de notación funcional a cifrado americano
5. Modelado
 - 5.1. Métricas para evaluación: top-k y MRR
 - 5.2. Modelos tradicionales
 - 5.2.1. Modelo Kneser–Ney (trigrama)
 - 5.2.2. Hidden Markov Model: utilizando funciones (T/S/D)
 - 5.2.3. Modelo Kneser-Ney (n-grama) con ajuste de hiper parámetros
 - 5.3. Redes Neuronales
 - 5.3.1. GRU
 - 5.3.2. LSTM
 - 5.3.3. LSTM (PyTorch) + ajuste de hiper parámetros
 - 5.4. Comparativa de modelos
 - 5.4.1. Métricas: top-k y MRR
 - 5.4.2. Comparación cualitativa
6. Re-ranking suavizado
 - 6.1. Penalizar repeticiones
 - 6.2. Incorporación de contexto musical extra
7. Despliegue en producción: FastAPI
 - 7.1. Endpoint principal: POST / Predict
8. Conclusiones y trabajo futuro
9. Bibliografía

(Anexos: código, tablas ampliadas, ejemplos de uso en:
github.com/antoniotrapote/chord-prediction-tfm)

1. Introducción y objetivos

La música, en el marco de la tradición occidental, puede analizarse como un lenguaje estructurado. De la misma manera que las palabras se organizan en frases, los acordes se combinan en progresiones que conforman la base armónica de una obra musical. Sin embargo, el dominio de la armonía funcional requiere años de estudio, y para muchos compositores o productores *amateur* constituye una barrera que dificulta su proceso creativo. Esta problemática abre un espacio de oportunidad para aplicar técnicas de ciencia de datos y aprendizaje automático al ámbito musical, con el objetivo de diseñar herramientas accesibles que faciliten el proceso creativo.

En este contexto surge el presente trabajo, que combina dos facetas complementarias. Por un lado, mi formación y experiencia previa como músico y musicólogo, lo que me ha permitido comprender en profundidad los retos asociados a la práctica compositiva. Por otro, la formación en minería de datos, *feature engineering*, modelado predictivo y despliegue de modelos adquirida durante el máster en *Data Science, Big Data & Business Analytics*. La **motivación principal** consiste en tender un puente entre ambas áreas para **crear un asistente de composición que facilite a cualquier usuario la generación de progresiones armónicas coherentes y musicalmente plausibles**.

El **objetivo general** del proyecto es **desarrollar un modelo predictivo capaz de sugerir acordes para completar una secuencia armónica dada**. Dicho modelo se entrenará sobre un conjunto de progresiones armónicas recopiladas en un dataset público, que se analizará y transformará para adaptarlo a las necesidades del proceso de modelización. El asistente resultante busca ser especialmente útil para músicos *amateur* que no dominan la teoría musical, actuando como una guía creativa que les ayude a explorar posibilidades musicales de forma sencilla.

Los **objetivos** específicos del trabajo son los siguientes:

1. Realizar un **análisis exploratorio del dataset** para comprender sus características principales (dimensión, distribución tonal, tipos de acordes, etc.).
2. Transformar los datos de entrada mediante procesos de parseo, normalización tonal y **transcripción de cifrado americano a notación funcional**, con el fin de disponer de una representación adecuada para los modelos.
3. Implementar y comparar diferentes enfoques de **modelado**: desde modelos probabilísticos clásicos (como **Kneser–Ney** o **Hidden Markov Models**) hasta arquitecturas neuronales recurrentes (**GRU** y **LSTM**).
4. Evaluar el rendimiento de los modelos con métricas de predicción secuencial (Top-k y Mean Reciprocal Rank).
5. Incorporar estrategias de **re-ranking** y filtros que optimicen la musicalidad de las sugerencias propuestas.
6. Despliegue del modelo mediante el desarrollo de una **API Rest** (FastAPI), que permita recibir progresiones de entrada y devolver predicciones en tiempo real.

La relevancia de este proyecto radica en su carácter interdisciplinar. Por un lado, se exploran paralelismos entre el procesamiento del lenguaje natural y el análisis de progresiones armónicas, considerando los acordes como “tokens” y las progresiones como “frases musicales”. Por otro, se busca aportar valor práctico con una herramienta que pueda integrarse en entornos creativos, como software de producción musical o aplicaciones docentes. Además, desde un punto de vista académico, el trabajo demuestra cómo un proceso *end-to-end* de ciencia de datos puede aplicarse a un dominio no convencional, poniendo de relieve tanto las capacidades como las limitaciones de las distintas técnicas.

Finalmente, el trabajo se estructura en siete capítulos principales. En primer lugar, se describe el origen del dataset y se lleva a cabo un análisis exploratorio de sus características. A continuación, se presentan las transformaciones realizadas para la preparación de los datos y la transcripción a notación funcional. El tercer bloque está dedicado al desarrollo y evaluación de distintos modelos, seguido de una sección donde se incorporan estrategias de re-ranking para refinar las predicciones. Posteriormente, se aborda el despliegue del modelo mediante el diseño de una API. El documento concluye con un capítulo de discusión y propuestas de mejora futura.

2. Origen y licencia del dataset

El dataset utilizado para la realización del proyecto está disponible públicamente en el repositorio de GitHub [“Jazz Chord Progressions Corpus”](#). Este corpus se desarrolló en el marco de la investigación de Bunks, Dixon, Weyde y Di Giorgi (2023), presentada en la conferencia International Society for Music Information Retrieval (ISMIR) en Milán (Bunks et al., 2023).¹

El corpus contiene **2.612 canciones**, que en conjunto suman **134.182 acordes**, de los cuales se identifican **1.532 símbolos únicos**. La mayor parte de las progresiones corresponden a estándares de jazz, aunque también se incluyen blues, jazz-blues, canciones tradicionales y algunos temas de música popular. Las progresiones de acordes están anotadas en cifrado americano (ej. *Cmaj7*, *Dm7*, *G7*), constituyendo así un recurso idóneo para la investigación de patrones armónicos y el entrenamiento de modelos de predicción.

En cuanto a su disponibilidad, el corpus se distribuye de manera abierta para fines académicos y de investigación, con la condición explícita de citar la publicación original (Bunks et al., 2023) al emplear los datos. Aunque el repositorio no incluye una licencia formal específica (como MIT o CC-BY), la puesta a disposición pública y la instrucción de atribución aseguran un uso legítimo del recurso, cumpliendo con la normativa general sobre derechos de autor y respetando las buenas prácticas en la reutilización de datasets académicos.

De esta forma, el corpus empleado no solo constituye un recurso de referencia en el ámbito de la Music Information Retrieval (MIR), sino que también asegura la transparencia y la trazabilidad necesarias para un proyecto académico como el presente.

3. Análisis exploratorio de los datos

3.1. Tonalidades (key)

En el corpus original, **únicamente hay etiquetadas tonalidades mayores**. Esto implica que las canciones originalmente compuestas en tonalidades menores aparecerán etiquetadas como su relativo mayor (Ej: 'Autumn Leaves', originalmente transcrita en sol menor -Gm-, aparece etiquetada como si bemol mayor -Bb-). Prestaremos especial atención a esto en la fase de transcripción funcional. Puesto que, si la tonalidad indicada no es correcta, no será posible la transcripción funcional de los acordes.

¹ Bunks, C., Dixon, S., Weyde, T., & Di Giorgi, B. (2023). *Modeling Harmonic Similarity for Jazz Using Co-occurrence Vectors and the Membrane Area*. In *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, Milan, Italy.

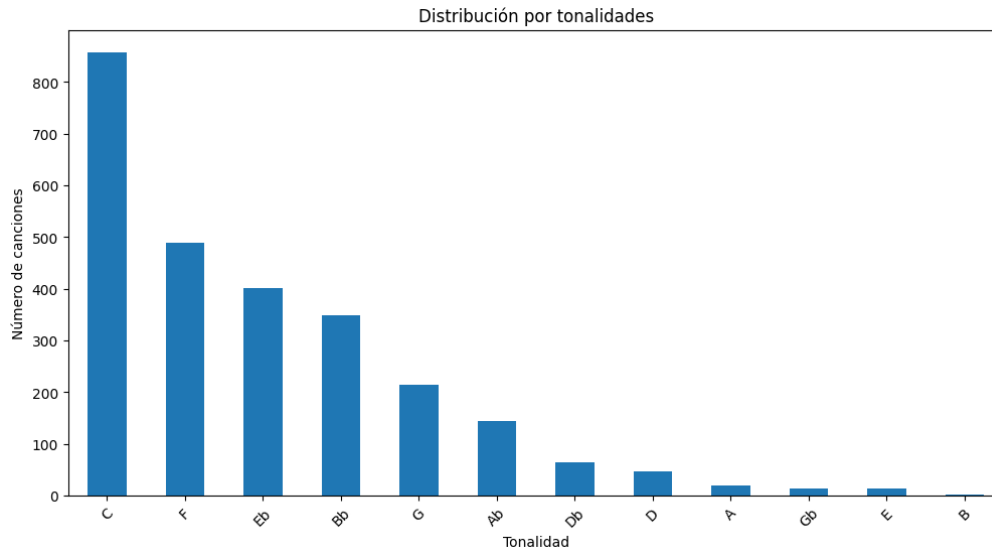


Fig.1. Distribución de canciones por tonalidad en el dataset original.

La desigualdad en el número de canciones que tenemos en el corpus se traduciría en un sesgo por tonalidades si utilizásemos el cifrado americano tal y como lo tenemos. Esto revela la importancia de hacer una transcripción funcional, que permita a nuestro modelo inferir relaciones armónicas independientemente de la tonalidad en la que esté la canción.

3.2. Métrica (*timesig*)

Observamos que el **91,58% de las canciones de nuestro corpus están en 4/4**.

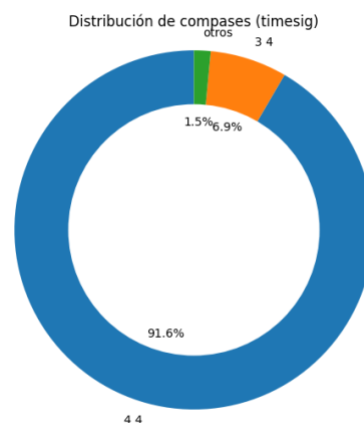


Fig.2. Distribución de canciones por tipo de compás en el dataset original.

Si nos limitáramos únicamente a las canciones en compás de 4/4, obtendríamos un conjunto de datos más sencillo de manejar, especialmente en lo referente a la representación temporal de los acordes (posición y duración). Ahora bien, la métrica no constituye un elemento imprescindible para el objetivo central de este trabajo —la sugerencia de acordes a partir de una progresión armónica—. Sin duda, su incorporación podría enriquecer el modelo, pero con el fin de mantener el proyecto dentro de un alcance viable, hemos decidido posponer este aspecto para futuras implementaciones, siempre y cuando el desarrollo actual muestre un recorrido prometedor.

3.3. Compases por canción (bars)

Si reparamos en el número de compases de cada canción, observamos que **el 43% de las canciones tienen concretamente 32 compases**. Esto refleja que nuestro corpus, está efectivamente compuesto principalmente por *standards* de jazz. Siendo 32 compases en una estructura AABA o similar la forma característica de este género.

En cualquier caso, la longitud de las progresiones no es de especial interés para nuestro propósito, puesto que las relaciones funcionales entre acordes de una progresión suelen establecerse entre un acorde y el inmediatamente siguiente generalmente. Nos detendremos en esta cuestión más adelante.

Lo que si puede resultar más relevante es que aproximadamente el **95 % de los registros tiene un número par de compases**. Esta regularidad es coherente con la organización formal típica (frases de 4, 8 o 16 compases) y sugiere que, de cara a futuros trabajos sobre estructura (p. ej., cadencias por secciones T/S/D), podríamos considerar excluir piezas con número impar de compases para estabilizar patrones cadenciales y periodicidad de las progresiones.

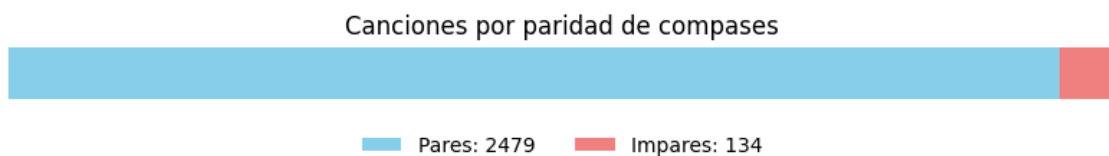


Fig.3. Distribución de canciones según la paridad de compases en el dataset original.

Por ahora conservaremos todas las canciones (con número par e impar de compases), dado que nuestros modelos trabajan sobre ventanas de acordes consecutivos y no dependen del número total de compases de cada canción.

3.4. Acordes (*chordprogresions*)



Fig.4. Nube de acordes más frecuentes en el dataset original.

Al inspeccionar el corpus original identificamos **1.532 símbolos de acordes únicos**. La nube de acordes permite apreciar de un vistazo una diversidad notable: encontramos acordes con bajo alternativo (ej.: Dm7/G), acordes con notas de color (6, 9, 13), acordes alterados (alt) o con la tercera suspendida (sus2, sus4), entre otras variantes. Esta heterogeneidad refleja la riqueza del lenguaje armónico en el jazz, pero también plantea un desafío desde el punto de vista del modelado automático.

Mostrar los **n-gramas más comunes**² en las progresiones de acordes revela que la progresión más común **II-V-I** en el corpus aparece en múltiples tonalidades diferentes. Para nuestro objetivo será imprescindible una capa de abstracción que nos permita identificar estas progresiones independientemente de la tonalidad. Esto nos lleva, de nuevo, a la necesidad de trabajar con notación funcional.

Si establecemos un **paralelismo con el procesamiento del lenguaje natural**, las progresiones de acordes pueden entenderse como secuencias de tokens. En este marco, **el parseo y normalización de acordes resulta un paso fundamental, equivalente a la lematización y tokenización en NLP**: unifica variantes y reduce la cardinalidad del vocabulario, lo que constituye un requisito indispensable para que los modelos sean capaces de generalizar correctamente.

En conclusión, el corpus en su forma original no es directamente adecuado para el entrenamiento de modelos predictivos. El sesgo hacia tonalidades mayores, la gran diversidad de cifrados y la existencia de grafías redundantes hacen imprescindible un proceso de parseo y normalización, que sentará las bases para obtener representaciones más estables y comparables en el modelado posterior.

[Para consultar el análisis exploratorio en detalle véase [chord-prediction-tfm/anexos/notebooks/01 análisis exploratorio/](#)]

4. Tokenización de los acordes: de cifrado americano a notación funcional.

El análisis exploratorio (Cap. 3) mostró dos retos para el modelado: sesgo tonal (solo claves mayores etiquetadas) y alta diversidad de grafías (más de 1.500 símbolos distintos). Para que nuestros modelos generalicen independientemente de la tonalidad y sin sobreajuste a variaciones de escritura, transformamos el cifrado americano en una **representación funcional estandarizada**.

Este proceso —que llamamos *tokenización de acordes*— tiene tres pasos: (4.1) parseo y normalización, (4.2) estimación de tonalidad, y (4.3) transcripción a notación funcional. Esta última aporta invariancia por transposición (una misma progresión se representa igual en cualquier clave).

4.1. Parseo y normalización de acordes

El primer paso consiste en convertir cada acorde del cifrado americano (original) a una tupla conformada por la nota fundamental del acorde y su cualidad, eliminando variaciones gráficas y agrupando cualidades en clases funcionales.

Ejemplos en C mayor:

- Cmaj7 > (0, maj7)
- Dm7 > (2, m7)

² Consultar anexos/notebooks/01_análisis exploratorio/02_edu.ipynb

4.1.1. De nombres de nota a enteros (*pitch class*)

Usamos dos alfabetos equivalentes y un diccionario enarmónico³:

```
PITCHES_SHARP = ["C", "C#", "D", "D#", "E", "F", "F#", "G", "G#", "A", "A#", "B"]
PITCHES_FLAT  = ["C", "Db", "D", "Eb", "E", "F", "Gb", "G", "Ab", "A", "Bb", "B"]
ENHARMONIC_ROOT = {"Cb": "B", "B#": "C", "Fb": "E", "E#": "F"}
```

Cualquier raíz textual se mapea a un entero 0–11: C=0, C#/Db=1, ..., B=11. Así C# y Db, por ejemplo, representan el mismo *pitch class* (1). **Este paso reduce el espacio de raíces a 12 enteros** (independientes de la grafía).

4.1.2. Normalización de cualidades (*quality*)

En primer lugar, agrupamos variantes de escritura a un alfabeto canónico:

```
QUAL_CANON = {
  "maj7": "maj7", "M7": "maj7", "Δ": "maj7", "maj": "maj", "M": "maj",
  "min": "m", "m": "m", "-": "m", "m7": "m7", "mMaj7": "mMaj7", "mM7": "mMaj7",
  "dim": "dim", "o": "dim", "o7": "dim7", "dim7": "dim7",
  "aug": "aug", "+": "aug",
  "7": "7", "9": "7", "11": "7", "13": "7",
  "ø": "m7b5", "m7b5": "m7b5", "halfdim": "m7b5",
  "sus2": "sus", "sus4": "sus", "sus": "sus",
  "6": "maj", "69": "maj", "add9": "maj"
}
```

Ejemplos: M7, Δ → maj7; ø → m7b5; 9/11/13 → 7 (dominante); 6/69/add9 → maj.

Si un acorde no especifica cualidad, aplicamos la heurística: si contiene “m” (y no “maj”), tratamos como m; en otro caso, maj.

En un segundo paso, para estabilizar el modelado secuencial, reducimos las 24 cualidades canónicas obtenidas en el paso anterior a 5 clases:

- **maj7** (mayor y variantes consonantes)
- **m7** (menor)
- **7** (dominante y variantes -aug, sus, extensiones-)
- **m7b5** (semidisminuido)
- **dim7** (disminuido)

En el siguiente gráfico (fig.5) podemos observar cómo algunas cualidades quedan “absorbidas” por la clase funcional reducida (*class_reduced*). En la categorización final categoría final ‘m7’ absorbe los acordes de triada (m) y cuatriada (m7) menores; la categoría ‘maj7’ absorbe de igual forma triadas (maj), cuatriadas (maj7) mayores y acordes mayores con tensiones (6, 9); “La categoría de dominantes aumenta porque absorbe acordes suspendidos (*sus*) y aumentados (*aug*). En definitiva, **las clases reducidas agrupan cualidades que cumplen funciones armónicas equivalentes**.

³ Enarmónico es el nombre que se le da a dos o más sonidos que a pesar de poseer distintos nombres, tienen una afinación, en la práctica, equivalente.

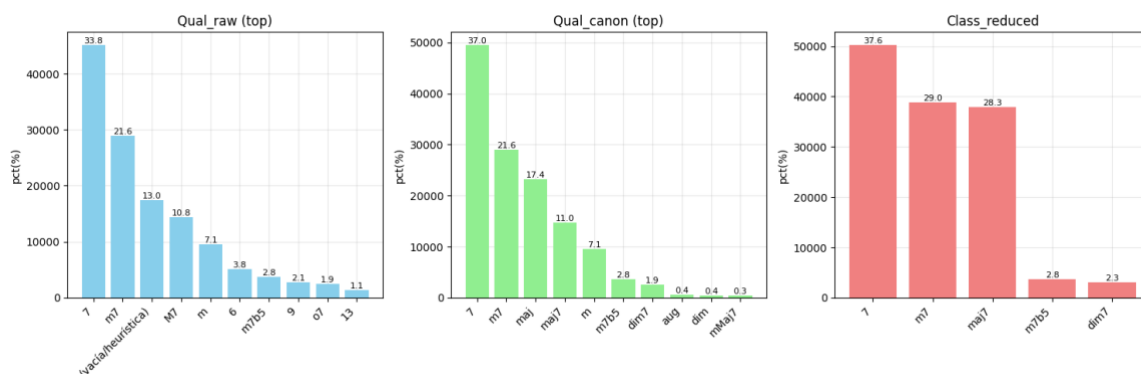


Fig.5. Distribución de acordes por tipo de cualidad en el dataset original: representación sin procesar (izquierda), cualidades canónicas (centro), y clases reducidas (derecha).

Reconocemos que al reducir el número de cualidades se pierden matices tímbricos y sonoros. Sin embargo, dado el público objetivo al que se orienta este asistente, consideramos adecuado conservar únicamente las cualidades mínimas necesarias para capturar la esencia funcional, evitando al mismo tiempo la explosión del vocabulario. Inversiones complejas o bajos alternativos tampoco se codifican explícitamente: optamos por representar solo la raíz, en una abstracción deliberada para favorecer la generalización.

4.2. Estimación de tonalidad

La transcripción a notación funcional requiere conocer previamente la tonalidad de la progresión. El corpus original solo etiqueta tonalidades mayores, y en algunos casos emplea la del relativo mayor para representar piezas en modo menor (véase el ejemplo de *Autumn Leaves* en el Capítulo 3). Para superar esta limitación, implementamos un procedimiento de **detección automática de tonalidad**.

El método se inspira en enfoques clásicos de análisis armónico: se evalúan las 24 tonalidades posibles (12 mayores y 12 menores), asignando a cada una puntuación en función de la compatibilidad entre los acordes de la progresión y los acordes esperados en dicha tonalidad.

Algoritmo de puntuación

1. **Mapeo a grados diatónicos.** Cada acorde parseado (pc, clase) se proyecta al grado relativo a la tonalidad candidata (ej.: Dm7 frente a C mayor $\rightarrow$ ii).
2. **Coincidencia de cualidad esperada.** Si la cualidad coincide exactamente con la esperada en ese grado (según tablas de acordes diatónicos para mayor y menor), se suman +2 puntos.
3. **Coincidencia parcial.** Si la cualidad no coincide exactamente, pero es razonablemente compatible (ej.: m7 frente a esperado m7b5, o maj frente a maj7), se asigna +1 punto.
4. **Cadencia final.** Si la progresión termina en una resolución $V \rightarrow I$, se añade un bono adicional de +1.5 puntos, reflejando la relevancia de la cadencia perfecta en la percepción tonal.
5. **Selección de tonalidad.** La tonalidad con mayor puntuación se selecciona como la más plausible. En caso de empate, se escoge la tonalidad mayor correspondiente.

Este procedimiento ofrece una estimación global para toda la canción. Sin embargo, muchas piezas presentan modulaciones parciales o alternancia entre mayor y menor

relativo. Para abordar estos casos con mayor precisión sería necesario implementar un algoritmo de segmentación tonal dinámica, capaz de detectar cambios de tonalidad dentro de una misma progresión. Esta línea se plantea como mejora futura (véase Capítulo 8).

4.3. Transcripción de acordes a notación funcional

Una vez parseados y normalizados los acordes, y estimada la tonalidad, el siguiente paso es su **transcripción a notación funcional**. Esta representación, basada en números romanos, permite expresar los acordes con relación a la tónica de la tonalidad y, por tanto, es **invariante por transposición**. Así, una progresión *ii-V-I* se representa de la misma manera en cualquier tonalidad.

Fundamento del método

- **Escalas de referencia.** Para establecer los grados diatónicos utilizamos las siguientes definiciones:

```
MAJOR_SCALE = [0,2,4,5,7,9,11]
MINOR_SCALE = [0,2,3,5,7,8,11]
ROMAN = ["I", "II", "III", "IV", "V", "VI", "VII"]
```

Donde ``MAJOR_SCALE`` y ``MINOR_SCALE`` contienen las posiciones (*pitch classes* relativos a la tónica) de cada grado.

- **Cualidades esperadas.** Se definen las cualidades armónicas diatónicas esperadas para cada grado, tanto en mayor como en menor:

```
EXPECTED_QUAL_MAJOR = {0:"maj7", 1:"m7", 2:"m7", 3:"maj7",
4:"7", 5:"m7", 6:"m7b5"}
EXPECTED_QUAL_MINOR = {0:"m7", 1:"m7b5", 2:"maj7", 3:"m7",
4:"7", 5:"maj7", 6:"dim7"}
```

Esto asegura que un acorde se interprete de acuerdo con su función tonal (p. ej., V7 en mayor, viiø en menor).

4.3.1. Acordes diatónicos

La función ``roman_for_chord()`` recibe un acorde parseado y devuelve su representación en números romanos:

- **Mayúsculas** para acordes mayores y dominantes.
- **Minúsculas** para menores, semi-disminuidos y disminuidos (con notación ø y o cuando corresponde).
- Añade el sufijo 7 si se trata de un dominante.

Ejemplo (en C mayor):

- Cmaj7 → I
- Dm7 → ii
- G7 → V7
- Bø → viiø

4.3.2. Préstamos habituales

Para acordes que no forman parte estricta de la escala, la función contempla **alteraciones cromáticas comunes**:

```
mapping = {1:"bII", 3:"bIII", 6:"#IV", 8:"bVI", 10:"bVII"}
```

En modo menor se añaden también la tercera y sexta “naturales” como préstamos del paralelo mayor (`natIII`, `natVI`)

Ejemplo (en C mayor): `Bbmaj7` → **bVII**

4.3.3. Dominantes secundarios y sustitutos

La detección de dominantes secundarios se realiza con la función ``_roman_secondary_or_sub()``.

1. Comprueba si el acorde actual es un **dominante (7)**.
2. Si resuelve en el siguiente acorde diatónico (distinto de la tónica), se etiqueta como **V/x** (dominante de grado x).
3. Si el acorde actual es el **sustituto por tritono** del dominante de un grado diatónico, se etiqueta como **Vsub/x**.

Ejemplos en C mayor:

- `A7 – Dm7` → **V/ii – ii**
- `D7 – G7` → **V/V – V7**
- `Db7 – C` → **Vsub/I – I**.

Este etiquetado enriquece el vocabulario funcional sin disparar el tamaño del corpus. Además, permite capturar uno de los recursos más característicos del jazz y la armonía funcional.

4.3.4. Ejemplo completo

Para la progresión: `Bm7b5 – E7 – Am – Dm7 – G7 – Cmaj7`

1. Detección tonal → C mayor.
2. Transcripción funcional:
 - `Bm7b5` → **viiø**
 - `E7` → **V/vi**
 - `Am` → **vi**
 - `Dm7` → **ii**
 - `G7` → **V7**
 - `Cmaj7` → **I**

Resultado: [**viiø, V/vi, vi, ii, V7, I**]

Síntesis

La transcripción funcional implementada:

- Garantiza **invariancia tonal**, permitiendo entrenar modelos sobre secuencias homogéneas.
- Integra no solo los grados diatónicos, sino también **préstamos cromáticos, dominantes secundarios y sustitutos**, ampliando el vocabulario de forma controlada.
- Reduce la complejidad del corpus original de **1.532 símbolos únicos a solo 82 tokens funcionales**, evitando la explosión de vocabulario y favoreciendo la capacidad de generalización de los modelos.
- Aporta la base teórica que conecta el corpus con los modelos de predicción, ya que la **función armónica** se convierte en el token principal de nuestro lenguaje musical.

4.4. Transcripción de notación funcional a cifrado americano

Aunque nuestro modelo trabaja internamente con notación funcional, para que la herramienta sea útil a un usuario final (p. ej., un compositor *amateur*), resulta imprescindible ofrecer la salida en cifrado americano estándar.

Implementamos por tanto una función de transcripción inversa que, a partir de la tonalidad estimada o indicada por el usuario, y el grado funcional predicho por el modelo, devuelve el acorde correspondiente en cifrado americano.

Principios del procedimiento

- Cada grado funcional se proyecta al *pitch class* absoluto de la tonalidad (ej.: ii en C mayor $\rightarrow$ D).
- La cualidad por defecto de cada grado se obtiene de las tablas de acordes diatónicos (ej.: ii en mayor $\rightarrow$ m7, V $\rightarrow$ 7, vii $\rightarrow$ $\emptyset$).
- El nombre de la nota se selecciona siguiendo la firma de la tonalidad (usando sostenidos o bemoles según corresponda).
- El acorde se devuelve en un formato legible y familiar: Dm7, G7, Cmaj7.

Ejemplo

- Entrada funcional: [ii, V7, I] en tonalidad de C mayor.
- Salida en cifrado americano: [Dm7, G7, Cmaj7].

Esta transcripción inversa no intenta recuperar todos los matices presentes en el corpus original (tensiones extendidas, inversiones, bajos alternativos). La decisión es deliberada: buscamos un equilibrio entre simplicidad y utilidad práctica. El usuario recibe acordes claros, coherentes y armónicamente válidos, que constituyen una base sobre la cual puede enriquecer o variar la armonización si lo desea.

[Para consultar los detalles de este proceso, y planteamientos anteriores a la solución final véase chord-predictin-tfm/anexos/notebooks/02_preprocesado]

5. Modelado

La tarea que abordamos en este proyecto -**predecir el siguiente acorde dado un contexto armónico**- presenta un paralelismo directo con la tarea clásica en modelado del lenguaje natural (NLP) de predecir la siguiente palabra en una secuencia. En ambos casos, se trabaja con un vocabulario discreto de tokens (palabras o acordes) y se busca estimar distribuciones de probabilidad sobre el siguiente elemento a partir de un historial de contexto.

Este paralelismo justifica tanto las métricas de evaluación adoptadas (Top-k, MRR), como la elección de modelos probabilísticos y neuronales que han demostrado eficacia en NLP: modelos de n-gramas con suavizado, modelos ocultos de Markov y redes neuronales recurrentes (GRU, LSTM). Aplicados a secuencias armónicas, estos métodos permiten capturar desde dependencias locales frecuentes (*ii-V-I*) hasta estructuras más complejas propias del jazz.

5.1. Métricas para evaluación: top-k y MRR

Tarea evaluada. Predicción del siguiente acorde dada una ventana de contexto (tokens funcionales). Para cada posición t en una progresión, el modelo produce un ranking de posibles acordes $a \in V$.

Top-k accuracy. Mide la proporción de veces en que el acorde verdadero a_t aparece entre las k primeras sugerencias:

$$\text{Top@k} = \frac{1}{N} \sum_{i=1}^N 1_{\{\text{rank}_i(a_t) \leq k\}}$$

- Top@1 $\equiv$ exactitud: acierta la primera sugerencia.
- Top@3, Top@5 capturan utilidad práctica cuando el usuario ve varias opciones.

Mean Reciprocal Rank (MRR).

El Reciprocal Rank (RR) para un ejemplo es el inverso de la posición del acorde correcto en el ranking:

$$RR_i = \frac{1}{\text{rank}_i(a_t)}$$

El **MRR** es el promedio sobre todos los ejemplos:

$$MRR = \frac{1}{N} \sum_{i=1}^N RR_i$$

- El mejor valor posible es **1.0** (siempre acierta en la primera posición).
- Cuanto **más cercano a 1, mejor es el modelo**.
- Cuanto más cercano a **0**, peor.

MRR penaliza suavemente cuando el acierto no es el primero y es sensible a mejoras en las primeras posiciones del ranking (lo que un usuario realmente ve).

Protocolo de evaluación

- **Partición** train/val/test del conjunto de progresiones.
- Evaluación **token a token** en test: para cada posición válida, se calcula el ranking y se acumulan Top@k y RR.
- En inferencia aplicamos el mismo **preprocesado** del Cap. 4 (clases funcionales, tokenización) y usamos los **tokens especiales** <bos> / </s> y <unk> (umbral de rareza).

5.2. Modelos tradicionales

Como primer paso evaluamos **modelos estadísticos clásicos**, óptimos para establecer un *baseline* sólido e interpretable. Estos métodos, ampliamente utilizados en modelado del lenguaje, permiten comprobar hasta qué punto la predicción de acordes **puede resolverse con dependencias locales simples**, antes de pasar a arquitecturas neuronales más complejas.

5.2.1. Modelo Kneser–Ney (trigrama)

Un **modelo n-grama** estima la probabilidad de un acorde dado un contexto limitado de longitud n-1. En el caso del trigrama, se calcula:

$$P(a_t | a_{t-1}, a_{t-2}) = \frac{C(a_{t-2}, a_{t-1}, a_t)}{C(a_{t-2}, a_{t-1})}$$

donde $C(\cdot)$ denota los conteos observados en el corpus.

El problema de esta formulación es que asigna probabilidad cero a los n-gramas no observados. Para evitarlo, aplicamos el **suavizado de Kneser–Ney (KN)**, que descuenta una constante D de los conteos y redistribuye la masa hacia modelos de menor orden. La probabilidad se interpola de la siguiente manera:

$$P_{KN}(a_t | h) = \frac{\max(C(h, a_t) - D, 0)}{C(h)} + \lambda(h) P_{KN}(a_t | h')$$

donde $h = (a_{t-2}, a_{t-1})$ es el historial, $h' = (a_{t-1})$ el contexto reducido, y $\lambda(h)$ un factor de normalización que asegura que las probabilidades sumen 1.

Este modelo es especialmente adecuado en armonía funcional, pues los patrones más frecuentes (como ii–V–I) se capturan dentro de la ventana de tres acordes, logrando un equilibrio entre simplicidad, rapidez de inferencia e interpretabilidad.

5.2.2. Hidden Markov Model: utilizando funciones (T/S/D)

A diferencia del modelo de n-gramas, que trabaja directamente sobre la secuencia de tokens, el modelo oculto de Markov (HMM) introduce un nivel intermedio: los **estados**

latentes que representan las funciones armónicas clásicas de la tonalidad —**Tónica (T)**, **Subdominante (S)** y **Dominante (D)**. La intuición es que las progresiones de acordes, aunque variadas en superficie, se organizan en torno a estas tres funciones, cuya sucesión sigue patrones relativamente estables (p. ej., $T \rightarrow S \rightarrow D \rightarrow T$).

Para entrenar el HMM fue necesario asignar a cada acorde funcional una etiqueta de función T, S o D. Implementamos una regla de mapeo basada en el grado romano y sus alteraciones:

- Grados **I, iii, vi** → función **Tónica**.
- Grados **ii, IV** → función **Subdominante**.
- Grados **V, vii** → función **Dominante**.
- Alteraciones frecuentes se interpretan según su rol típico: **bII, bVI, bVII** como S, **bIII** como T.

De este modo, la secuencia observable de acordes se convierte en una sucesión de emisiones condicionadas a los estados ocultos T, S o D. Formalmente, el modelo se define por:

- Una matriz de transición $A = P(s_t | s_{t-1})$ con $s_t \in \{T, S, D\}$.
- Una matriz de emisión $B = P(a_t | s_t)$ que asigna probabilidades a cada acorde en función de su función armónica.

El entrenamiento se realiza mediante máxima verosimilitud, y la predicción del siguiente acorde se obtiene a partir de la distribución del estado latente más probable en el tiempo $t + 1$.

Este modelo ofrece **alta interpretabilidad**, al permitir inspeccionar directamente las transiciones entre funciones y las distribuciones de emisión de cada una. Sin embargo, su **capacidad predictiva es limitada**: al operar en el nivel de funciones (T, S, D), no distingue con precisión entre acordes concretos que comparten la misma función, lo que reduce su utilidad práctica frente a modelos que trabajan directamente sobre la secuencia de tokens.

5.2.3 Modelo Kneser-Ney (n-grama) con ajuste de hiper parámetros

A partir del trigramma, exploramos **contextos de distinto orden** (3, 4) y **distintos valores de descuento D** (0.5, 0.7, 0.75, 0.9, 1.0). La búsqueda de hiper parámetros se orienta a maximizar la métrica **MRR** en el conjunto de validación.

Formalmente, el modelo generaliza como:

$$P_{KN}(a_t | a_{t-n+1}^{t-1}) = \frac{\max(C(a_{t-n+1}^{t-1}, a_t) - D, 0)}{C(a_{t-n+1}^{t-1})} + \lambda(a_{t-n+1}^{t-1}) P_{KN}(a_t | a_{t-n+2}^{t-1})$$

donde a_{t-n+1}^{t-1} representa el historial de longitud $n - 1$

Este ajuste nos permite comparar configuraciones y observar cómo varía el rendimiento en función de la longitud de contexto. En la práctica, los resultados mostraron que órdenes intermedios (trigramma y 4-grama) ofrecen el mejor compromiso entre **precisión** y **generalización**.

5.3. Redes Neuronales

Los modelos tradicionales analizados (KN y HMM) ofrecen interpretabilidad y ligereza computacional, pero presentan limitaciones:

- El KN se restringe a ventanas de contexto fijas (trigramas o n-gramas),
- El HMM reduce la armonía a tres funciones abstractas (T, S, D), perdiendo detalle a nivel de acorde.

Las **redes neuronales recurrentes (RNNs)** surgen como alternativa natural. En el ámbito del NLP han demostrado su capacidad para **capturar dependencias de largo alcance** y aprender representaciones distribuidas de los tokens. En el terreno del lenguaje musical, esperamos que estas arquitecturas sean capaces de:

- Reconocer progresiones típicas que exceden el alcance de un trígama o tetragrama (p. ej., secuencias AABA, cadencias prolongadas).
- Diferenciar entre acordes concretos dentro de la misma función armónica.
- Mejorar la calidad del ranking al tener en cuenta **información contextual más amplia**.

El entrenamiento de estos modelos exige **recursos computacionales superiores**. Por ello recurrimos a Google Colab, aprovechando aceleración por GPU, y utilizamos **PyTorch**, un framework que proporciona módulos optimizados para RNNs y facilita el entrenamiento, validación y comparación de distintos modelos.

5.3.1. GRU

La **GRU** (Cho et al., 2014)⁴ es una variante de las redes neuronales recurrentes diseñada para trabajar con secuencias. A diferencia de una red neuronal tradicional (MLP), que procesa ejemplos de manera independiente, la GRU mantiene un **estado oculto** que se actualiza en cada paso, transmitiendo información del contexto anterior. Gracias a sus compuertas (*reset* y *update*), actúa como una **memoria selectiva** capaz de decidir qué información conservar y cuál descartar, mitigando además el problema del *vanishing gradient* que afecta a las RNN simples. Esto la hace especialmente adecuada para tareas musicales, donde el valor de un acorde solo cobra sentido en relación con los que lo preceden. Su menor complejidad respecto al LSTM la convierte además en un punto de partida razonable para evaluar el potencial de las RNN en este problema.

5.3.2. LSTM

El **LSTM** (Hochreiter & Schmidhuber, 1997)⁵ es la variante más conocida de RNN. Introduce una celda de memoria y tres compuertas (entrada, olvido y salida), lo que le permite capturar **dependencias de largo alcance** en secuencias musicales. En nuestro contexto, la LSTM puede retener información relevante durante varios compases (p. ej., cadencias prolongadas, cadenas de *ii-V-I* o turnarounds de 8/16 compases) y refinar la predicción del siguiente acorde a partir de un historial amplio. Aunque en teoría ofrece mayor capacidad expresiva que la GRU, a costa de más parámetros y mayor coste computacional, la literatura no es concluyente respecto a cuál de las dos arquitecturas

⁴ Cho, K., et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*.

⁵ Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation.

resulta superior en la práctica. Por ello, decidimos entrenar ambas variantes y comparar empíricamente su rendimiento en nuestro corpus.

5.3.3. LSTM (PyTorch) + ajuste de hiper parámetros

Dado que las primeras pruebas con LSTM ofrecieron métricas ligeramente superiores a las de la GRU, planteamos un **pipeline de ajuste de hiper parámetros** con el objetivo de explorar si era posible mejorar su rendimiento. Este proceso incluyó la variación de parámetros clave como la dimensión de los embeddings, el número de capas recurrentes, el tamaño de la capa oculta, la tasa de *dropout* y la longitud de secuencia utilizada como contexto.

El ajuste se llevó a cabo mediante una búsqueda aleatoria (***random search***) sobre un espacio de configuraciones predefinido, evaluando cada modelo entrenado en el conjunto de validación con las métricas descritas en 5.1 (Top@k y MRR). Esta estrategia nos permitió balancear la limitación de recursos computacionales (entrenamiento en Google Colab con GPU) con la necesidad de explorar suficientes combinaciones como para detectar tendencias relevantes.

En síntesis, este apartado nos permitió comprobar en qué medida la calidad de las predicciones dependía de la arquitectura y de su configuración interna. Más allá de los resultados concretos —que se discuten en 5.4—, este experimento puso de relieve la importancia de un ajuste sistemático como paso necesario para evaluar con rigor la capacidad de un modelo en contextos musicales complejos.

5.4. Comparativa de modelos

Este proyecto ha seguido un **proceso iterativo**: distintos métodos de transcripción/normalización dieron lugar a versiones de dataset diferentes. Para evitar confusiones y facilitar la trazabilidad, preservamos en los anexos las métricas más relevantes obtenidas por cada modelo sobre los dos últimos datasets consolidados.⁶

Dado que las diferencias entre las métricas de algunos modelos resultaron muy ajustadas, complementamos el análisis cuantitativo con una evaluación cualitativa basada en criterios musicales (Sección 5.4.2), con el fin de valorar la utilidad práctica del ranking de sugerencias en situaciones reales de composición.

5.4.1. Métricas: top-k y MRR

Para asegurar una comparación justa, todos los modelos comparten la misma tokenización funcional (Cap. 4), el mismo vocabulario efectivo y el mismo protocolo de evaluación (Sección 5.1: Top@1/3/5 y MRR).

La partición de datos difirió según la naturaleza del modelo: se utilizó partición por canción en los modelos tradicionales (KN, HMM) para evitar fuga de información entre fragmentos de una misma obra, mientras que en las redes neuronales (GRU, LSTM) se trabajó con secuencias completas tokenizadas, lo que responde a la forma en que estas arquitecturas procesan el contexto.

⁶ Para consultar los distintos preprocesados evaluados véase ../anexos/notebooks/02_preprocesado

Los resultados se presentan en tablas que recogen, para cada modelo, las métricas **Top@1**, **Top@3**, **Top@5** y **MRR**, calculadas sobre el mismo conjunto de posiciones de test. Esto nos permitió comparar de manera directa la precisión y la calidad de ranking alcanzada por cada enfoque.

En síntesis:

- **Kneser–Ney** ofreció un **baseline sólido y competitivo**. La versión ajustada (n-grama con tuning de hiperparámetros) mostró **mejoras moderadas**, en especial en Top@1 y MRR.
- **HMM (T–S–D)** se situó claramente por debajo en todas las métricas, confirmando que la reducción de la armonía a tres funciones abstractas implica pérdida de granularidad y menor capacidad de acierto en el acorde concreto.
- **Redes neuronales (GRU, LSTM)** lograron métricas **comparables a KN** con varias configuraciones. En conjunto, las diferencias observadas entre los mejores modelos (KN ajustado vs. LSTM) fueron **pequeñas y sin un patrón robusto**, por lo que evitamos afirmar superioridad concluyente.

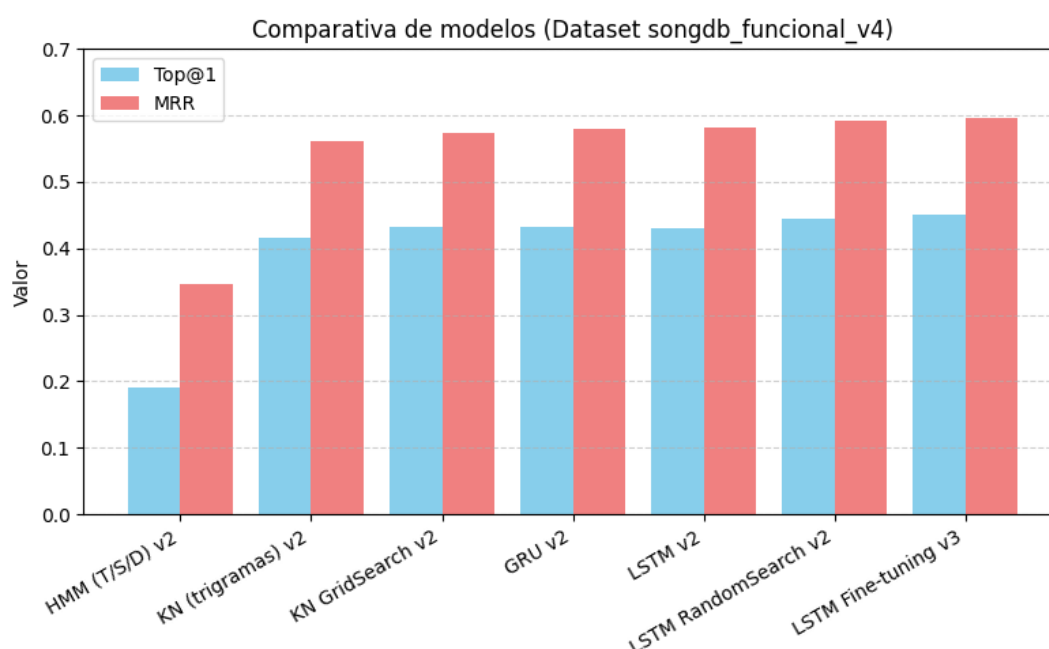


Figura 6. Comparativa de modelos entrenados con *songdb_funcional_v4*. Se muestran los valores de Top@1 (precisión exacta) y MRR (calidad de ranking) para cada modelo.

5.4.2. Comparación cualitativa

Las métricas globales (Top@k, MRR) mostraron diferencias poco concluyentes entre Kneser–Ney y las redes neuronales recurrentes. Para contrastar su utilidad en situaciones musicales concretas, realizamos una **evaluación cualitativa** diseñada ad hoc.

Definimos una *suite* de pruebas compuesta por progresiones características en diferentes contextos:

- **Modo mayor:** cadencias auténticas, patrones pop I–V–vi–IV, intercambios modales, uso del bVII.

- **Modo menor:** progresiones $ii\bar{o}-V7-i$, resoluciones desde VI, preparación del dominante.
- **Secundarios y sustitutos:** dominantes secundarios, sustitutos por tritono, cadenas de dominantes.
- **Cold-start y rarezas:** arranques sin contexto o con un solo acorde inicial.

Este protocolo nos permitió contrastar **qué tan plausibles** eran las sugerencias en un marco armónico musicalmente justificado, más allá de la métrica estadística.

Para cuantificarlo se usaron dos indicadores adicionales:

- **Hit rate:** proporción de veces que el acorde esperado aparecía en el *top-k*.
- **Plaus_avg:** número medio de sugerencias musicales plausibles, valoradas según criterios musicales.

Los resultados generales se muestran a continuación. Las tablas completas están disponibles en el repositorio.

=== RESUMEN POR SUITE ===								
	suite	cases	KN_hits	LSTM_hits	KN_mean_rank	LSTM_mean_rank	KN_plaus_sum_avg	LSTM_plaus_sum_avg
0	Cold-start y rarezas	3.0	3.0	0.0	1.667	NaN	3.000	0.000
1	Modo mayor	7.0	6.0	4.0	1.667	2.25	1.429	0.714
2	Modo menor	3.0	3.0	2.0	1.000	1.50	1.000	0.667
3	Secundarios y sustitutos	5.0	5.0	4.0	1.400	1.00	1.200	1.000
=== SCORES GLOBALES ===								
	KN_plaus_avg	LSTM_plaus_avg	KN_hit_rate	LSTM_hit_rate				
0	1.556	0.667	0.944	0.556				

Tabla 2. Resultados cualitativos comparativos entre Kneser–Ney y LSTM en suites de prueba representativas. Se reportan la tasa de acierto (*Hit rate*) y el número medio de sugerencias plausibles (*Plaus_avg*).

A pesar de que el LSTM ofrecía métricas ligeramente mejores en validación (Top@k, MRR), en las pruebas cualitativas con progresiones armónicas reales se observa que **el modelo Kneser–Ney:**

- Tiene una **tasa de acierto mucho más alta** (94% vs 56%).
- Ofrece **sugerencias más plausibles y consistentes** en todos los contextos musicales evaluados.
- Es estable, explicable y determinista, lo que justifica su elección como modelo final para despliegue en producción (implementación en la API descrita en el capítulo 6).

[Para consultar los detalles sobre modelado y comparativa de modelos véase chod-prediction-tfm/anexos/notebooks/03_modelado]

6. Re-ranking suavizado

El modelo Kneser–Ney ofrece sugerencias razonables, pero observamos cierta tendencia a proponer repeticiones consecutivas entre los candidatos (ej.: V7 seguido de otro V7). Desde un punto de vista musical, y teniendo en cuenta que el modelo no incorpora todavía información temporal, estas redundancias no aportan valor a nuestro asistente de composición.

Con el fin de afinar los resultados, **diseñamos un módulo de re-ranking que reevalúa la lista de predicciones y ajusta sus probabilidades**. Este paso no sustituye al modelo base, sino que lo complementa, mejorando la diversidad de las sugerencias y acercando el sistema a un prototipo funcional.

Además, desde la perspectiva de la experiencia de usuario, consideramos útil implementar filtros adicionales que permitan restringir las sugerencias a acordes estrictamente diatónicos en una tonalidad dada, o bien ofrecer un modo más flexible que incorpore préstamos y dominantes secundarios, aportando mayor versatilidad al producto final.

6.1. Penalizar repeticiones

El módulo de *re-ranking* incorpora un **mecanismo anti-repetición** para evitar que el sistema proponga de nuevo acordes que ya han aparecido en la ventana de contexto inmediato. Este comportamiento se implementa con dos parámetros:

- **rep_window**: tamaño de la ventana de contexto (en nuestro caso, 2).
- **alpha_repeat**: intensidad de la penalización, con valores entre 0 y 1 (en nuestro caso, 0.25).

De este modo, si un acorde candidato ya está presente en la ventana de los últimos acordes, su probabilidad se atenúa según:

$$P'(a_t) = \begin{cases} (1 - \alpha_{\text{repeat}}) \cdot P(a_t), & \text{si } a_t \in \text{ventana reciente} \\ P(a_t), & \text{en otro caso.} \end{cases}$$

Finalmente, todas las probabilidades se re-normalizan para conservar una distribución válida y se reordena el ranking. Esto permite que el sistema conserve la información estadística original del modelo KN, pero favoreciendo una mayor diversidad y evitando redundancias poco útiles para el usuario final.

6.2. Incorporación de contexto musical extra

Además de la penalización de repeticiones, el módulo de *re-ranking* integra funciones de filtrado basadas en la teoría musical:

- **_is_diatonic()**: comprueba si un acorde pertenece a la escala de referencia (incluyendo la excepción del **bVII** en modo menor).
- **_is_secondary()**: identifica dominantes secundarios o sustitutos por tritono.
- **_has_accidental()**: identifica acordes con alteraciones cromáticas respecto a la escala.

La función rerank aplica estas utilidades según el **modo de filtrado** seleccionado:

- **Free** → conserva todas las predicciones del modelo base.
- **Diatonic** → solo admite acordes diatónicos en la tonalidad, descartando o atenuando el resto.
- **Functional_plus** → admite, además de los diatónicos, dominantes secundarios y sustitutos.

De este modo, el *re-ranking* no sustituye al modelo KN, sino que reorganiza sus salidas en función de reglas musicales explícitas, adaptando el comportamiento al nivel de flexibilidad seleccionado por el usuario final.

[Para consultar los detalles sobre la implementación del re-ranking véase [chord-prediction-tfm/anexos/notebooks/04_reranking](https://chord-prediction-tfm.github.io/anexos/notebooks/04_reranking)]

7. Despliegue en producción: FastAPI

Para cerrar el ciclo de un proyecto *end-to-end*, desarrollamos una API con FastAPI que permite consultar el modelo en tiempo real. Este demuestra la viabilidad práctica del sistema y su potencial uso como asistente de composición musical. La API convierte el prototipo en una herramienta accesible para cualquier aplicación externa (por ejemplo, un editor de partituras, un plugin DAW o una aplicación web)

7.1. Endpoint principal: POST / predict

El endpoint recibe una secuencia en cifrado americano y parámetros opcionales para tonalidad y re-ranking y devuelve sugerencias para el siguiente acorde.

Campos de entrada

- `sequence (str)`: progresión de entrada en cifrado americano (espacios/comas/barritas).
- `k(int)`: nº de sugerencias (top-k).
- `tonic (int, opc.)`: tónica como *pitch class* 0–11 (C=0,...,B=11). Si se omite, se infiere la tonalidad.
- `mode (str, opc.)`: "major" o "minor". Si se omite, se infiere la tonalidad.
- `filter_mode (str)`: "free", "diatonic" o "functional_plus".
- `alpha_repeat (float)`: penalización de repeticiones (0–1).
- `rep_window (int)`: ventana reciente para detectar repeticiones.
- `beta_filter (float)`: atenuación "suave" si `hard_filter=false`.
- `hard_filter (bool)`: `true` elimina acordes no permitidos; `false` solo los atenúa.

Respuesta (JSON, campos principales).

- `detected_key`: si proporcionas `tonic/mode`, `is_manual=true` y `confidence="manual"`. Si no, contiene la tonalidad inferida y una puntuación de confianza.
- `Roman_sequence`: transcripción funcional del contexto
- `predictions`: lista ordenada de acordes sugeridos y probabilidad tras el **re-ranking**.

[Para consultar los detalles de la API véase el archivo `app.py` en el directorio principal del repositorio [chord-prediction-tfm](https://chord-prediction-tfm.github.io) . Instrucciones de despliegue local en `README.md`]

8. Conclusiones y trabajo futuro

El presente trabajo abordó el reto de diseñar un asistente de composición musical capaz de predecir acordes en progresiones armónicas. Partiendo de un corpus público, desarrollamos un pipeline *end-to-end* que incluyó: análisis exploratorio de datos, normalización y transcripción a notación funcional, entrenamiento de distintos modelos (estadísticos y neuronales), re-ranking de predicciones y despliegue en una API mediante FastAPI.

Los resultados muestran que un modelo estadístico clásico, el interpolado de Kneser–Ney, ofrece un rendimiento competitivo frente a arquitecturas más complejas como GRU o LSTM. A pesar de que las redes neuronales lograron métricas ligeramente superiores en validación, la evaluación cualitativa en contextos musicales concretos reveló que KN produce sugerencias más plausibles y musicales. Este hallazgo es relevante: un modelo ligero, con bajo coste computacional y sin dependencias externas, puede superar o igualar a sistemas más sofisticados en una tarea práctica de predicción armónica. Además, las predicciones de un modelo estadístico de este tipo son perfectamente explicables, frente a la opacidad de modelos de *caja negra* como las redes neuronales.

La incorporación de un módulo de re-ranking permitió afinar las recomendaciones, penalizando repeticiones y filtrando según diferentes grados de restricción tonal. Esto aporta flexibilidad de uso: desde una exploración libre hasta un modo estrictamente diatónico, ajustándose a distintos perfiles de usuario. Finalmente, la implementación de una API convierte el modelo en una herramienta accesible, integrable en aplicaciones musicales reales (editores de partituras, plugins para DAWs o entornos educativos).

El trabajo presenta, sin embargo, varias limitaciones. El corpus utilizado está centrado en estándares de jazz, lo que condiciona la representatividad del modelo. En cuanto a los modelos neuronales, se exploraron en profundidad arquitecturas recurrentes (GRU y LSTM, incluyendo ajuste de hiperparámetros), pero no se abordaron arquitecturas más recientes como Transformers o modelos preentrenados en música, que podrían ofrecer mejoras adicionales. Tampoco se consideró la dimensión temporal rítmica, que podría enriquecer la predicción armónica en contextos más amplios.

Como líneas de trabajo futuro proponemos:

- Ampliar y diversificar el corpus con repertorios de otros géneros y estilos.
- Incorporar segmentación tonal más fina (cambios de tonalidad, modulaciones locales).
- Explorar arquitecturas modernas (Transformers, modelos preentrenados en música).
- Enriquecer la interfaz de la API, por ejemplo, con *endpoints* que permitan aislar las funcionalidades de transcripción y predicción, o incluyendo un manejo de errores consistente.
- Validar el sistema con usuarios reales (compositores, estudiantes, docentes), integrando su *feedback* en el ciclo de desarrollo.

En conclusión, este proyecto demuestra que es posible construir un asistente de predicción armónica funcional y eficiente a partir de técnicas accesibles, combinando teoría musical y modelos de lenguaje. Más allá de los resultados, constituye una prueba del potencial de aplicar métodos de NLP al análisis y generación de estructuras musicales, abriendo un camino para desarrollos futuros tanto en investigación como en aplicaciones creativas.

Bibliografía

- Bunks, C., Dixon, S., Weyde, T., & Di Giorgi, B. (2023). Modeling Harmonic Similarity for Jazz Using Co-occurrence Vectors and the Membrane Area
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft). Stanford University.
- Lerdahl, F. (2001). *Tonal pitch space*. Oxford University Press, USA